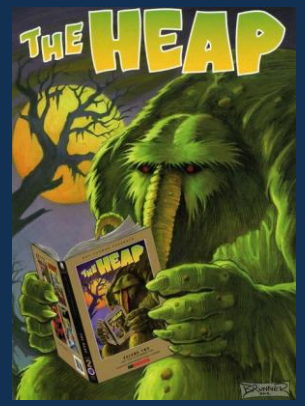
02. Dynamic Memory Allocation

Kyu-haeng Lee

SW Convergence Coding II

Fall 2025



DKU CI Lab

Dynamic Storage Allocation

- C's data structures are normally fixed in size.
  - Static data structures must have their sizes decided at time of compilation
    - char cities[10];
  - May waste memory.

	0	1	2	3	4	5	6	7	8	9
cities[0]	P	u	n	e	\0					
cities[1]	M	u	m	b	a	i	\0			
cities[2]	D	e	l	h	i	\0				
cities[3]	N	a	g	p	u	r	\0			

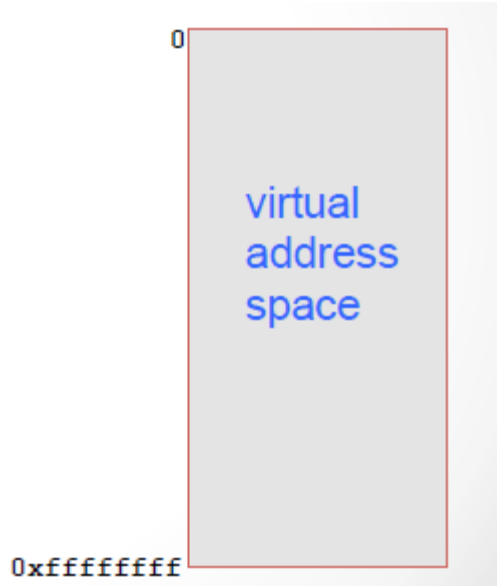
Dynamic Storage Allocation

- Fortunately, C supports dynamic storage allocation: the ability to allocate storage during program execution.
  - used most often for strings, arrays, and structures.
  - done by calling a memory allocation function.

Memory Layout

- What is stored in memory?
  - Code
  - Constants
  - Global and static variables
  - Local variables
  - Dynamic memory (malloc)

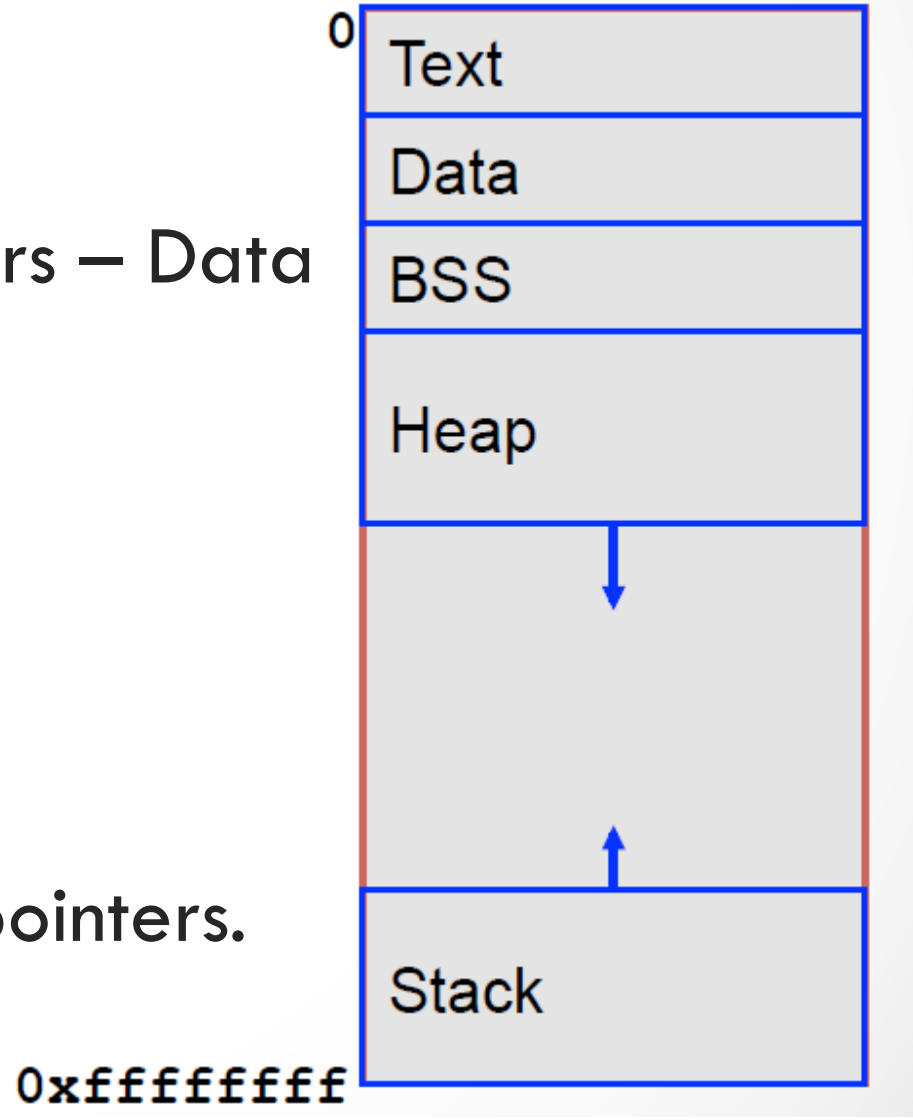
```
int SIZE;                          ← global
char* f(void) {
    char* c;                       ← local
    SIZE = 10;
    c = malloc(SIZE);              ← dynamic
    return c;
}
```



Memory Layout

- How is memory organized?
  - Code – Text
  - Constants, initialized global and statics vars – Data
  - Uninitialized global and static vars – BSS
  - Dynamic memory (malloc) – Heap
  - Local variables – Stack

- The Heap
  - Heap memory is only accessible through pointers.



Memory Allocation Functions

- Memory allocation related functions
  - malloc: allocates a block of memory without initialization
  - calloc: allocates a block of memory with initialization
  - realloc: resizes a previously allocated block of memory.

  - free: releases the previously allocated memory

- <stdlib.h> header should be included

Memory Allocation Functions

- malloc: allocates a block of memory without initialization
  - void *malloc (size_t size);

```
int* p = malloc(sizeof(int) * 10);
//memory blocks for 10 ints are allocated.

for (int i = 0; i < 10; i++)
    p[i] = i;
```

  - Allocates a block of “size” bytes from the heap
    - size_t is an unsigned integer type.
  - Returns a pointer to the block allocated
    - A cast is not required in C but is required in C++.

Memory Allocation Functions

- free: releases previously allocated memory blocks
  - void free (void *ptr);
  - ptr should point to the first block of the memory.

```
int* p = malloc(sizeof(int) * 10);
//memory blocks for 10 ints are allocated.

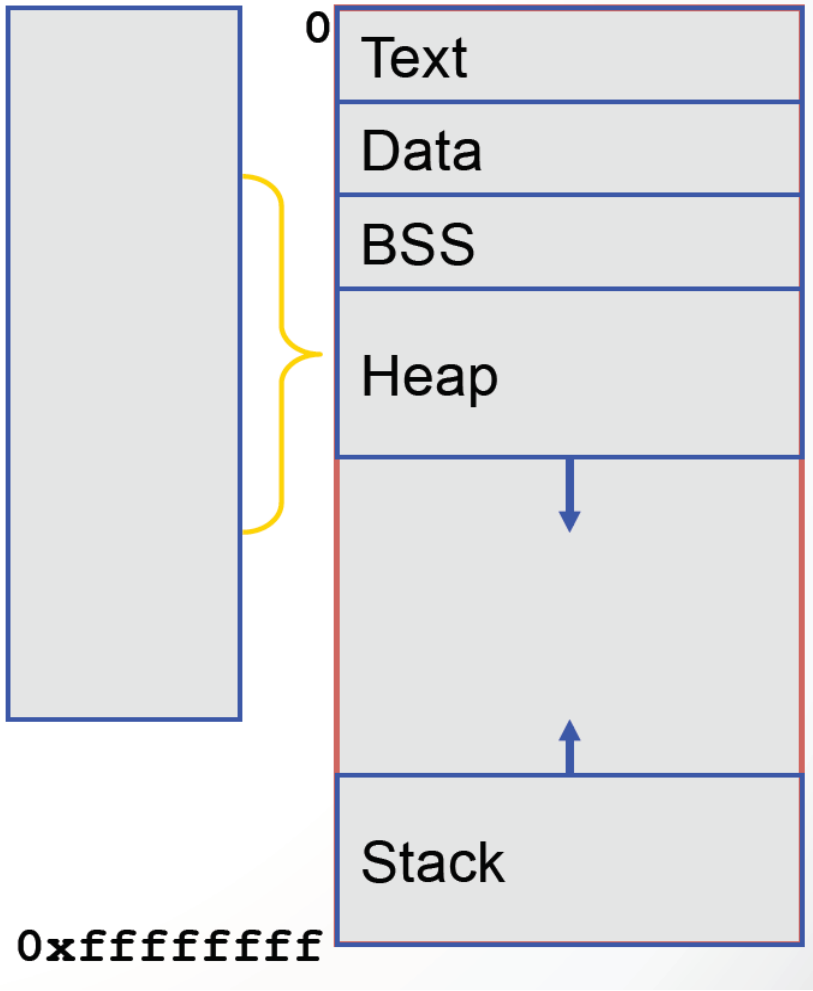
for (int i = 0; i < 10; i++)
    p[i] = i;

free(p);
```

  - The whole memory blocks of the pointer, allocated by malloc and calloc, are freed.

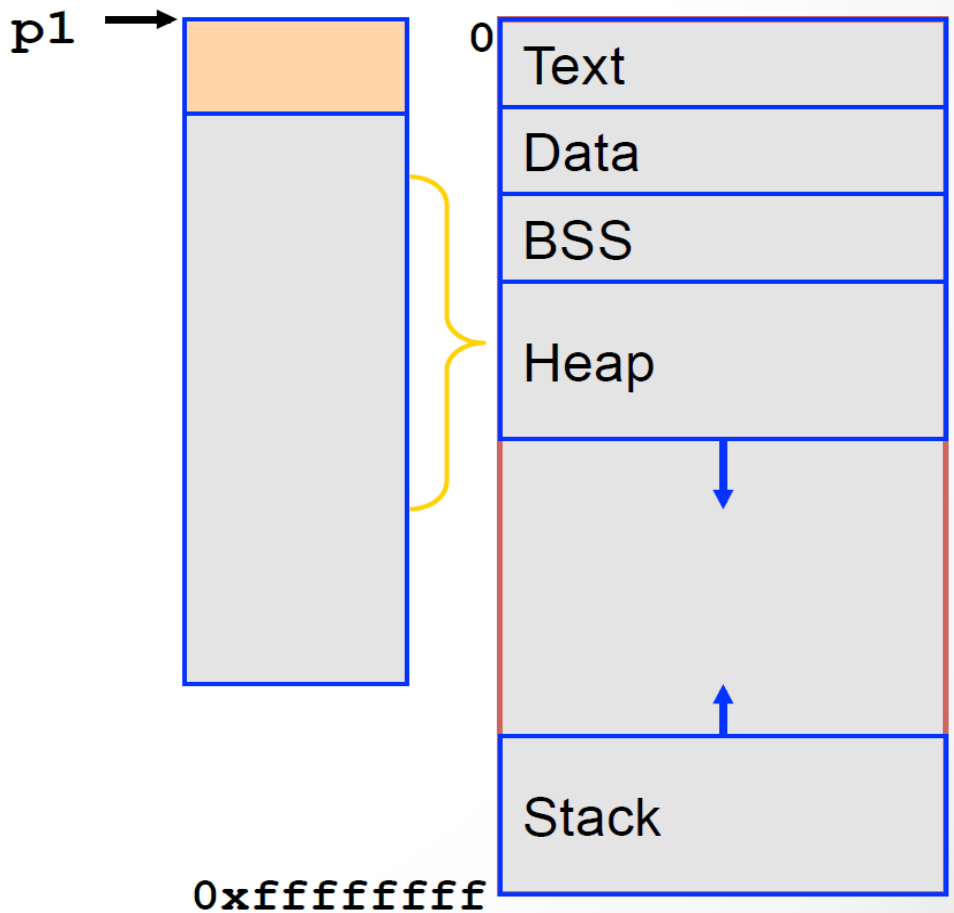
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



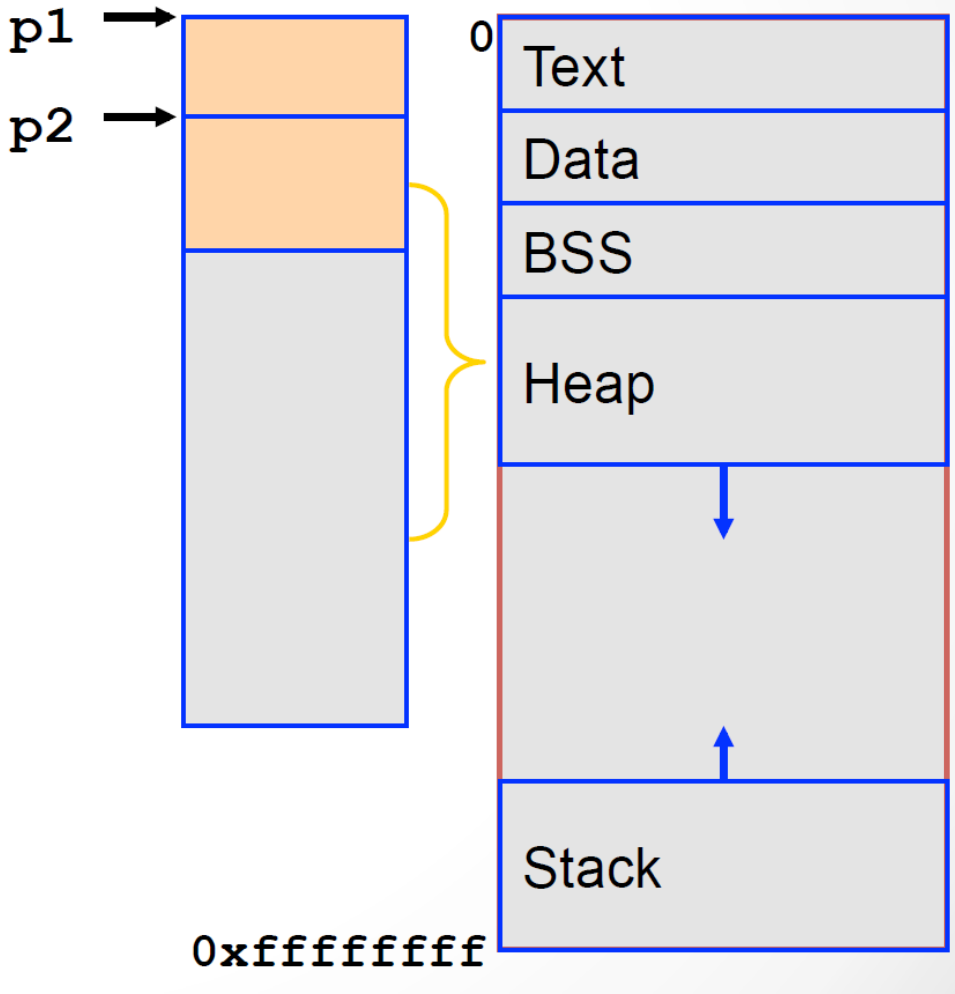
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



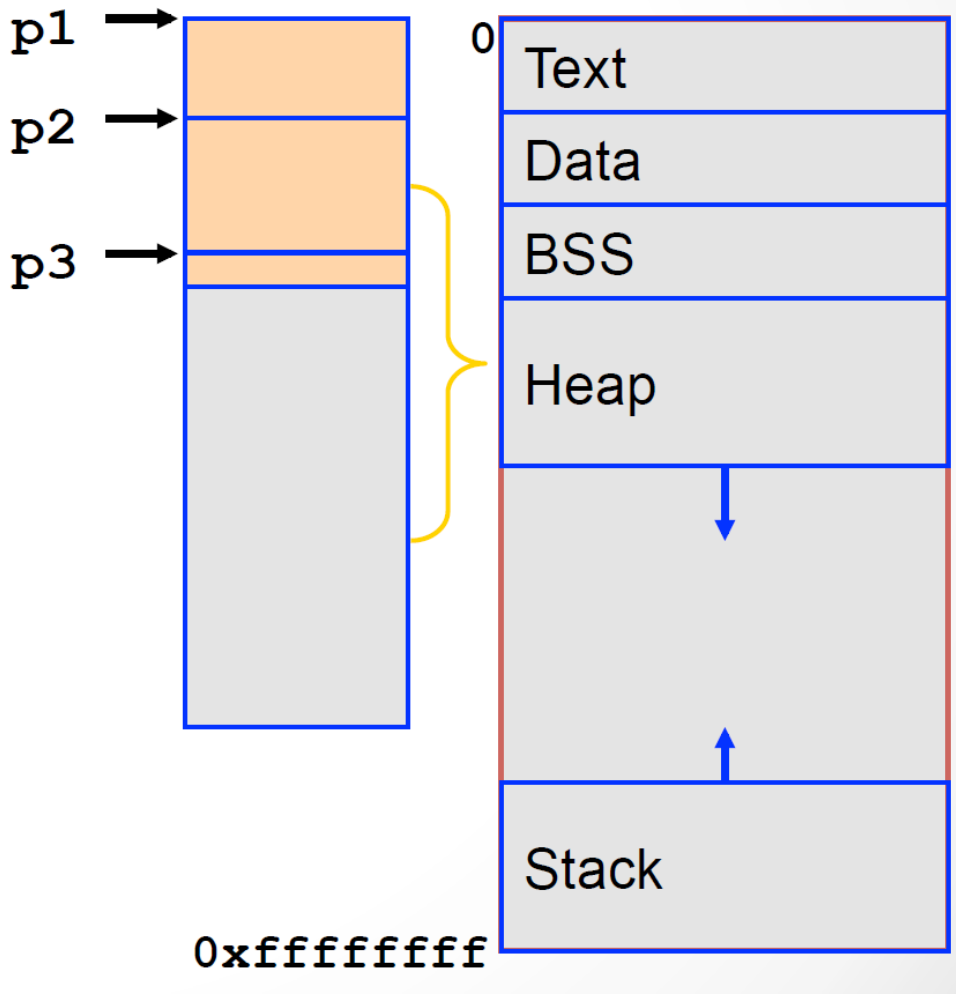
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



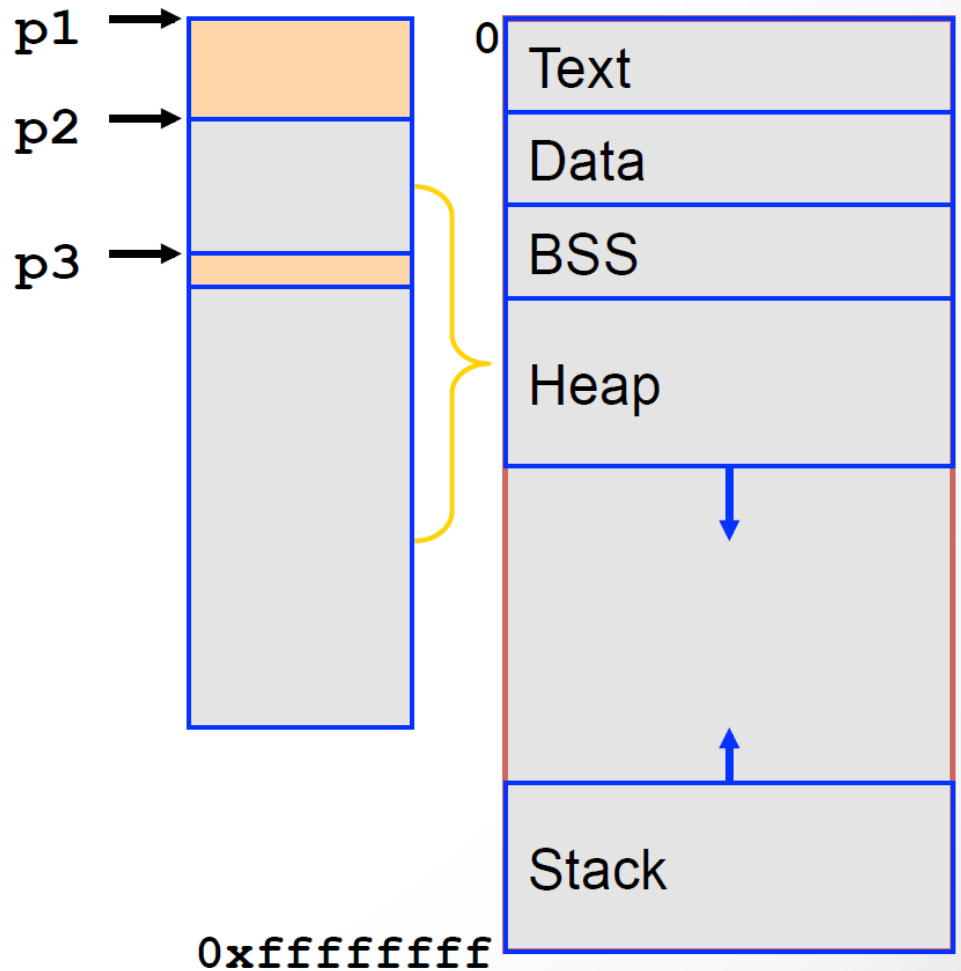
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



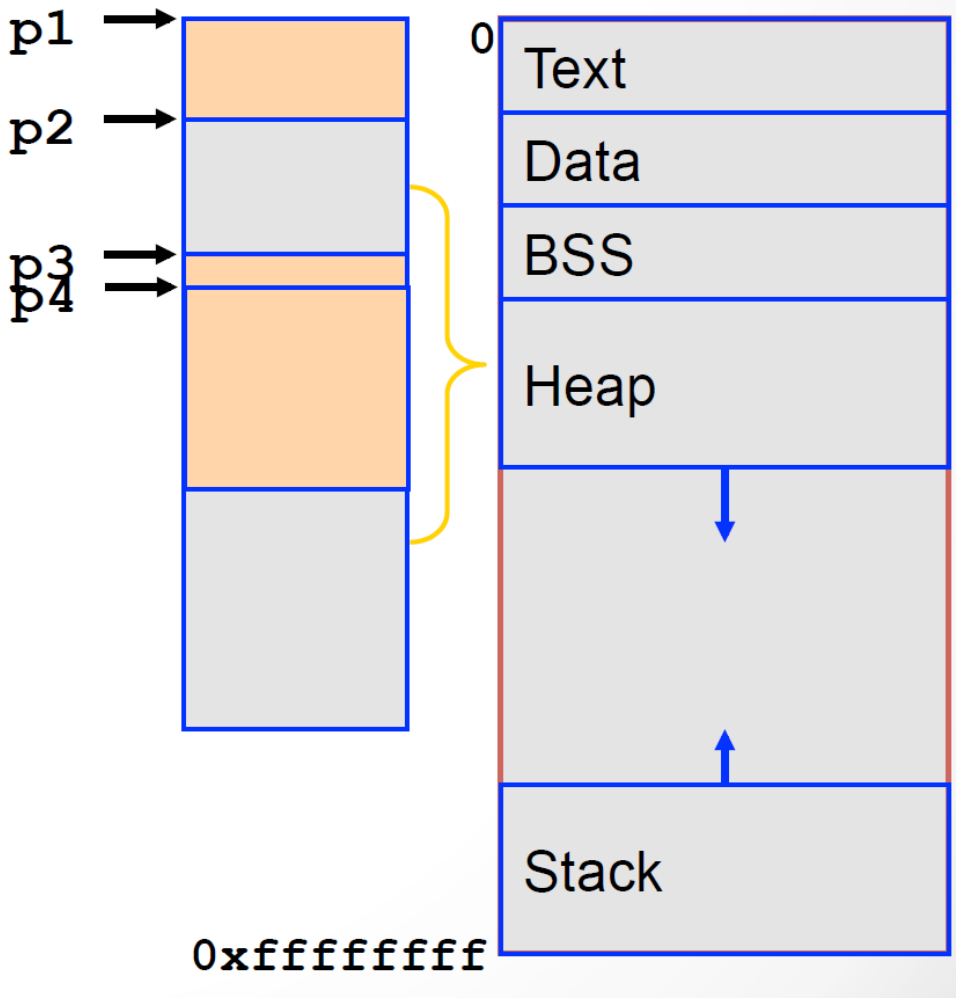
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



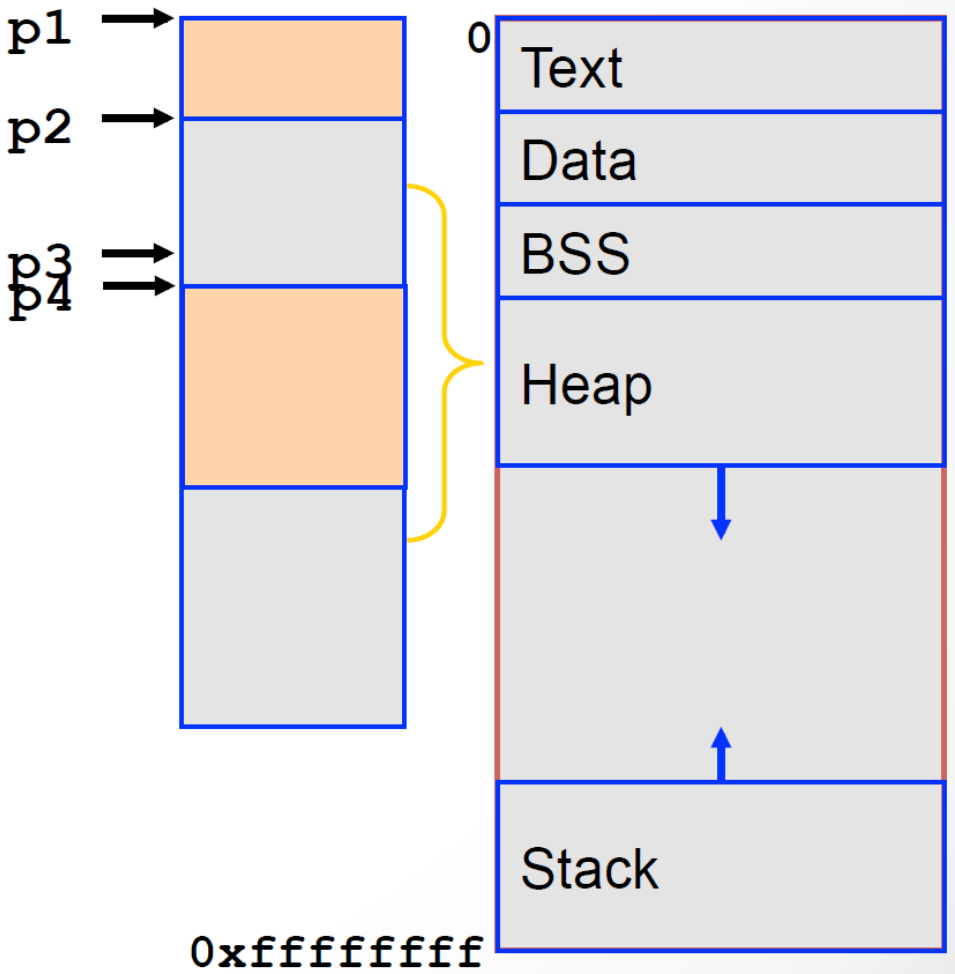
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



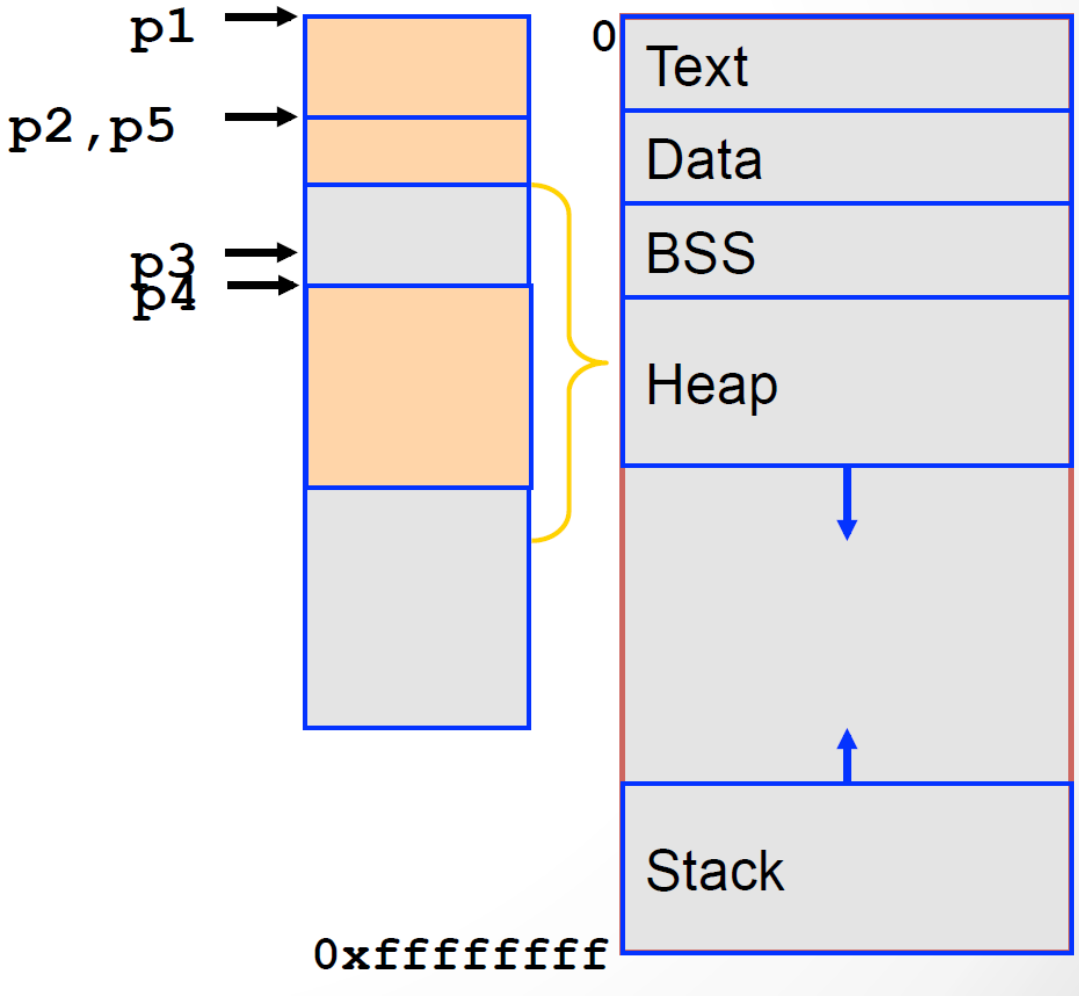
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



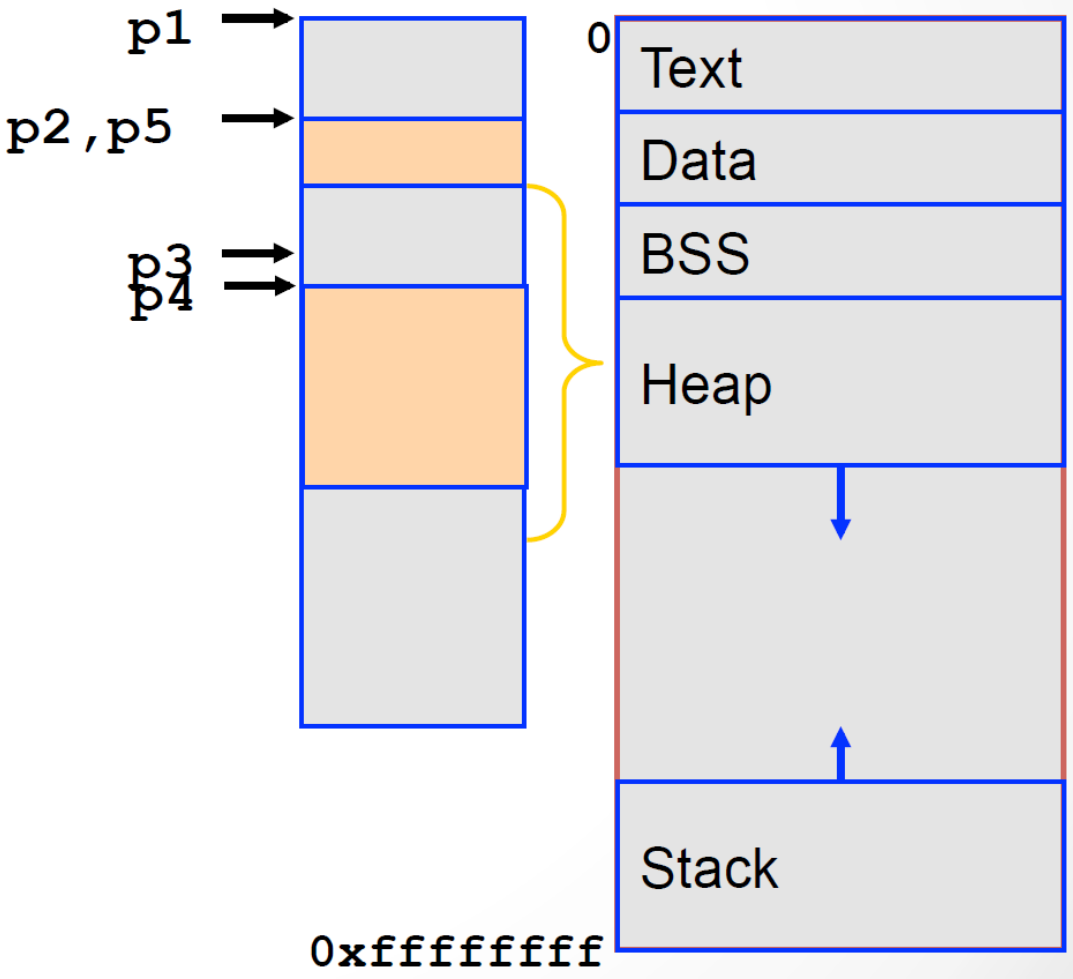
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



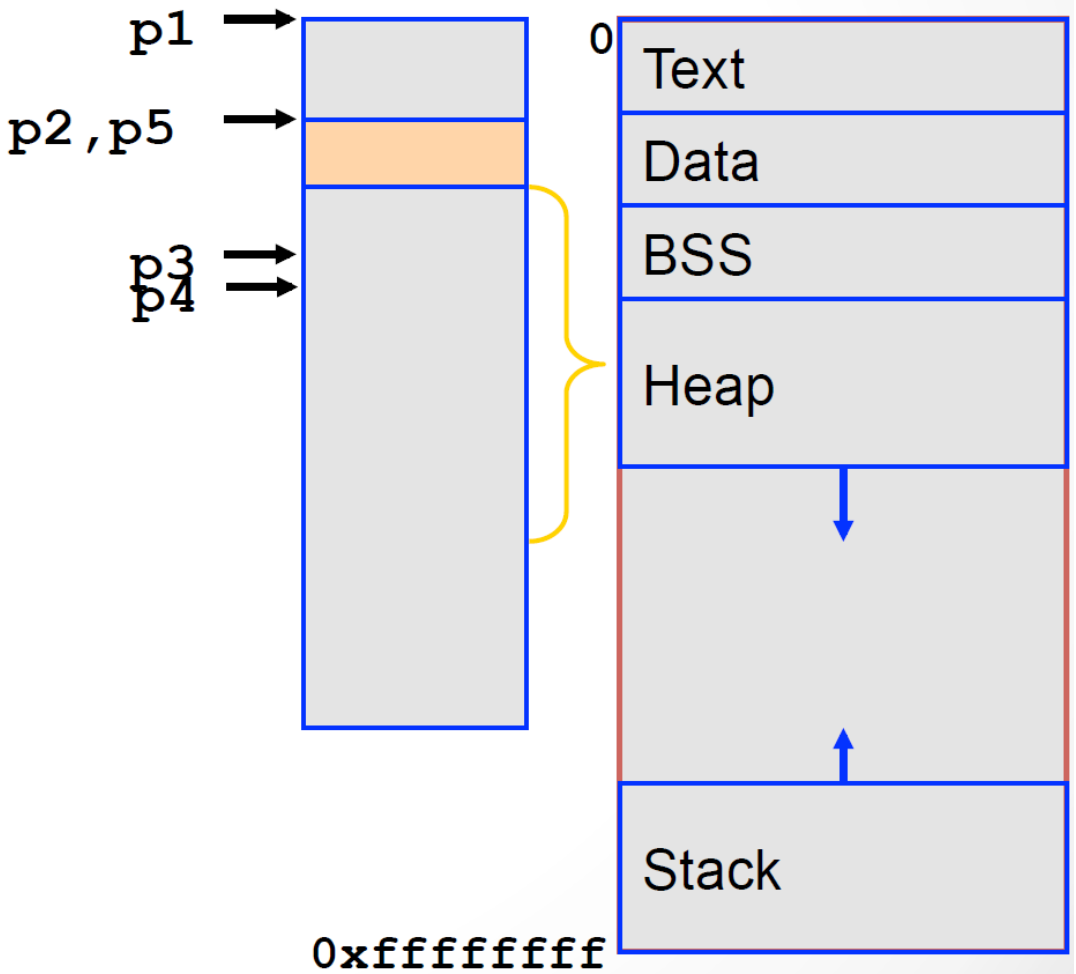
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



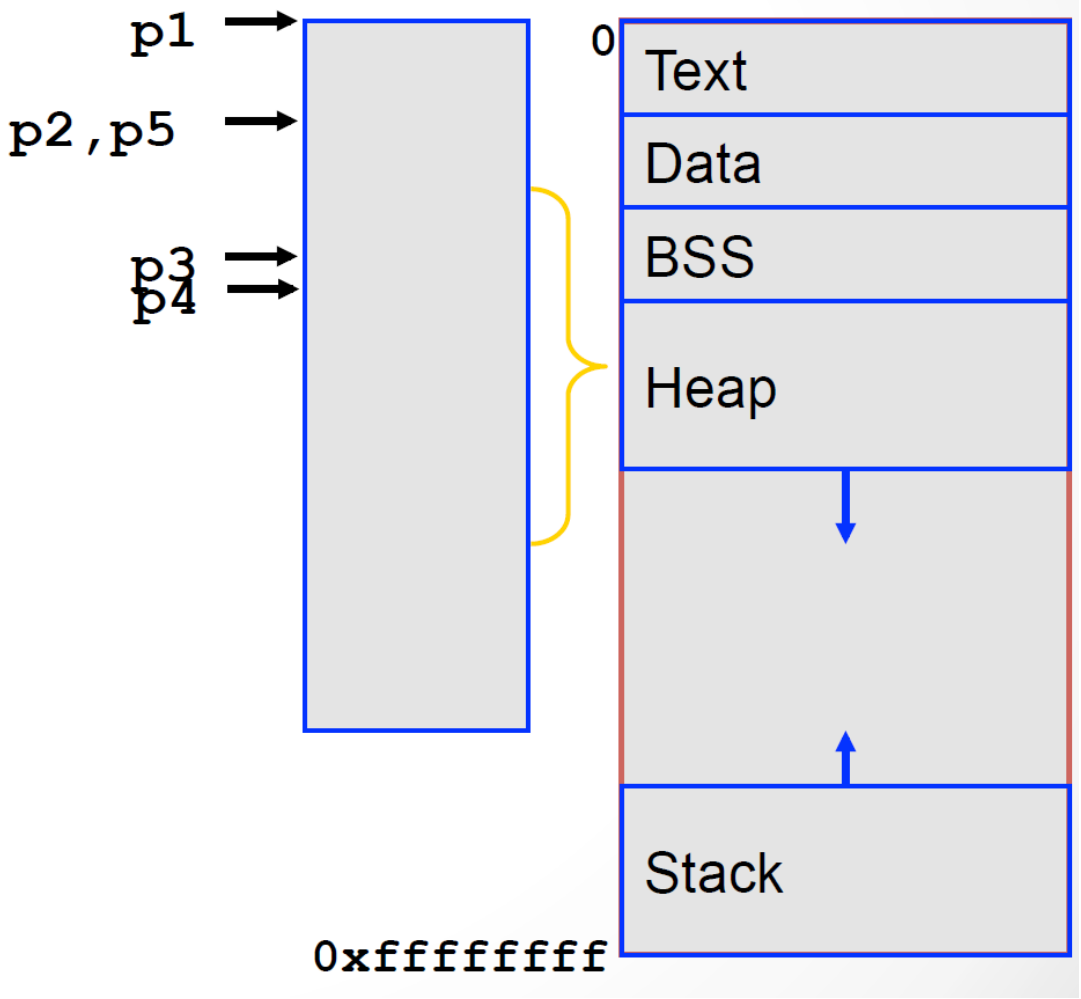
Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



Dynamic Memory Layout

```
char *p1 = malloc(3);
char *p2 = malloc(4);
char *p3 = malloc(1);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



Example: int Array Allocation

- Suppose a program needs an array of n integers, where n is computed during program execution.

```
int n;
scanf("%d", &n);

int* a = malloc(sizeof(int) * n);

int i;
for (i = 0; i < n; i++)
   a[i] = i;

free(a);    //free the memory
```

Exercise (09/22)

```c
#include <stdio.h>
// TO DO 1. Include the header to use malloc and free.

// TO DO 2. Provide a function my_arr.
// my_arr allocates memory for an array of "num" ints through dynamic memory allocation.

int main() {
    int num = 0;
    scanf("%d", &num);
    int* p = my_arr(num);

    for (int i = 0; i < num; i++) {
    //TO DO 3. Assign some values to the array.
    }

    for (int i = 0; i < num; i++) {
    //TO DO 4. Print the values of the array.
    }

    //TO DO 5. Free the memory.

    return 0;
}
```

Memory Allocation Functions

- calloc: allocates a block of memory and initializes it with zeros.
  - void *calloc (size_t number, size_t size);

```
int* p = calloc(10, sizeof(int));
//memory blocks for 10 ints are allocated.
//and initialized to zeros.

for (int i = 0; i < 10; i++)
   printf("%d₩n", p[i]);
```

Memory Allocation Functions

- realloc: resizes a previously allocated block of memory.
  - void *realloc (void *memblock, size_t size);
  - memblock: pointer to the target memory to be resized.
  - size: new size

```
int* p = calloc(5, sizeof(int));

//if the following code is commented out,
an error occurs.
p = realloc(p, 10 * sizeof(int));

for (int i = 0; i < 10; i++)
    p[i] = i;
```

  - NULL as 1st argument → malloc.
  - 0 as 2nd argument → free.

Memory Allocation Functions

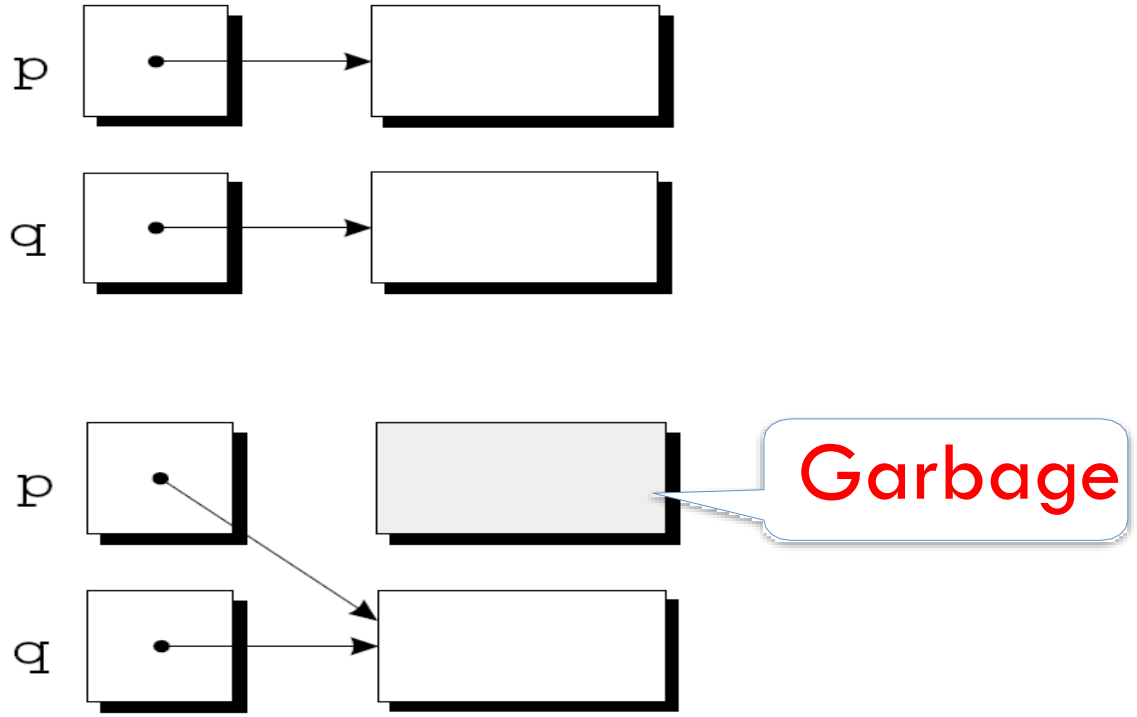
- If a memory allocation function can't locate a memory block of the requested size, it returns a null pointer.

```
p = malloc(10000);
if (p == NULL) { //same as: if (!p)
   /* allocation failed;
      take appropriate action */
}
```

- A null pointer is a special value that can be distinguished from all valid pointers.
- After we've stored the function's return value in a pointer variable, we must test to see if it's a null pointer.

Deallocating Storage

- Consider
  - p = malloc(…);
  - q = malloc(…);
  - p = q;



- Both variables point to the second memory block:
  - A program that leaves garbage behind has a memory leak.
  - Calling the free function to release unneeded memory.

The “Dangling Pointer” Problem

- Dangling pointers
  - free(p) deallocates the memory block that p points to, but doesn’t change p itself.
  - If we forget that p no longer points to a valid memory block, chaos may ensue:

```
char *p = malloc(4);
free(p);
strcpy(p, "abc");          /*** WRONG ***/
```



The “Dangling Pointer” Problem

- How to handle dangling pointers.
  - Stack variables (i.e. locals in functions) are deallocated as soon as a function returns. Therefore, avoid assigning pointers to addresses of such variables.

  - Assign NULL to the pointer, right after calling the free.
    - free(p);
    - p = NULL;

Example: 2D Array

- The array size is determined at run time. (N by M)
- Memory for an array of pointers is first allocated.
- How to Free?
  - free(ptr); ??

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int N, M;
    scanf_s("%d %d", &N, &M);
    int** ptr = malloc(N * sizeof(int*));
    for (int i = 0; i < N; i++) {
        ptr[i] = malloc(M * sizeof(int));
    }

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            ptr[i][j] = i * M + j;
        }
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            printf("%d₩t", ptr[i][j]);
        }
        printf("₩n");
    }
    return 0;
}
```

Example: 2D Array

- Free
  - The reverse order of allocation
  - Free all (int*)s by going though the loop.
  - free(ptr) frees only the array of pointers.

```
int N, M;
scanf_s("%d %d", &N, &M);

int** ptr = malloc(N * sizeof(int*));
for (int i = 0; i < N; i++) {
    ptr[i] = malloc(M * sizeof(int));
}

for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        ptr[i][j] = i * M + j;
    }
}

for (int i = 0; i < N; i++) {
    free(ptr[i]);
}
free(ptr);
```

References

- Chapter 14. Other C Topics, C: How to Program, Paul Deitel et al.